



Pickelton Backend Project Plan (Java - Spring Boot)



Goal (MVP)

Build a production-ready backend where:

- Users authenticate
 - Clubs exist
 - Tournaments are created
 - Players register
 - Matches are played
 - Leaderboard works
-



System Modules

auth

user

club

tournament

registration

match

leaderboard

config

security

common



Team Structure



Rule

- One person controls DB schema (Team Lead)
 - Others follow schema + API contract strictly
-



Person 1 — Auth + User + Club



Modules

- auth

- user
- club

Tables

users

clubs

club_members

Tasks

Auth

- User entity
- Register API
- Login API
- Password hashing (BCrypt)
- JWT token generation
- JWT filter (Spring Security)

User

- Get profile API
- User DTO

Club

- Create club
- Join club
- List clubs
- Role system (ADMIN / MEMBER)

APIs

POST /auth/register

POST /auth/login

GET /users/me

POST /clubs

POST /clubs/{id}/join

GET /clubs

✓ Output

- Working authentication system
 - Users stored in DB
 - Club system functional
-

Person 2 — Tournament + Registration

Modules

- tournament
- registration

Tables

tournaments

registrations

Tasks

Tournament

- Tournament entity
- Create tournament
- List tournaments
- Get tournament details
- Validate:
 - creator = user
 - optional club_id

Registration

- Register for tournament
- Prevent duplicate registration
- Slot limit validation
- Registration status

APIs

POST /tournaments

GET /tournaments

GET /tournaments/{id}

POST /tournaments/{id}/register

✓ Output

- Tournament system working
 - Registration system working
 - Slot validation implemented
-

Person 3 — Match + Leaderboard

Modules

- match
- leaderboard

Tables

matches

Tasks

Match

- Create match
- Assign players
- Update score
- Set winner
- Add round field

Leaderboard

- Compute wins/losses
- Rank players
- Tournament leaderboard API

APIs

POST /matches

PATCH /matches/{id}/score

GET /tournaments/{id}/leaderboard

✅ Output

- Matches working
 - Score updates working
 - Leaderboard functional
-

👑 Team Lead Responsibilities (You)

🔧 Tasks

🏠 Setup

- Create Spring Boot project
- Setup folder structure
- Setup DB connection (Supabase)
- Initialize repo

🗄 Database

- Define schema
- Maintain consistency
- Approve changes

🔗 Integration

- Merge modules
- Fix API mismatches
- Ensure full flow works

🚀 Deployment

- Deploy backend
 - Configure environment variables
 - Test production APIs
-

🔗 Dependency Flow

Auth → User → Club

↓

Tournament → Registration

↓

Match → Leaderboard

Timeline

Week 1 — Foundation

- Auth complete
- DB setup done
- Club APIs ready

Week 2 — Core

- Tournament APIs
- Registration system

Week 3 — Gameplay

- Match APIs
- Leaderboard

Week 4 — Final

- Integration
 - Deployment
 - Bug fixing
-

Git Workflow

main → stable

dev → integration

feature/auth

feature/tournament

feature/match

Rules

- One DB schema owner
- Same naming conventions
- Use DTOs (no direct entity exposure)
- Test APIs daily

- Integrate early
-

Risks

- Auth delay blocks everything
 - DB mismatch breaks integration
 - API mismatch causes failures
-

MVP Scope

Build Now

- Auth
- Clubs
- Tournaments
- Registration
- Matches
- Leaderboard

Later

- Payments
 - Feed system
 - Image uploads
 - Analytics
-

Final Product Vision

Pickelton =

Auth

+ Clubs

+ Tournaments

+ Matches

+ Leaderboard

Later:

+ Feed

+ Payments

+ Analytics



PICKELTON FRONTEND PLAN (Next.js)

Goal

Build a **scalable web app (Next.js)** that:

- connects to your Spring Boot backend
- supports tournaments, matches, clubs
- is structured for team development
- can later extend to mobile (Expo)



TECH STACK

- **Framework** → Next.js (App Router)

- **Language** → TypeScript
- **Styling** → Tailwind CSS
- **API calls** → Axios / fetch
- **State (basic)** → React state (later Zustand if needed)

🌱 PROJECT STRUCTURE (FINAL)

pickelton-web/

```

├── app/
│   ├── (auth)/
│   │   ├── login/page.tsx
│   │   └── signup/page.tsx
│   ├── dashboard/page.tsx
│   ├── clubs/
│   │   ├── page.tsx
│   │   └── [id]/page.tsx
│   ├── tournaments/
│   │   ├── page.tsx
│   │   ├── create/page.tsx
│   │   └── [id]/page.tsx
│   ├── matches/
│   │   ├── page.tsx
│   │   └── [id]/page.tsx
│   ├── leaderboard/
│   │   └── [id]/page.tsx
│   ├── profile/page.tsx
│   └── layout.tsx
├── components/
├── services/
└── hooks/

```

└─ types/

└─ utils/

└─ styles/

CORE PAGES (MVP)

Auth

- /login
- /signup

Dashboard

- /dashboard
- Shows:
 - upcoming tournaments
 - quick actions

Clubs

- /clubs
- /clubs/[id]

Features:

- view clubs
- join club

Tournaments

- /tournaments
- /tournaments/create
- /tournaments/[id]

Features:

- list tournaments
- create tournament
- view details

- register
-

Registration

👉 inside tournament page

Matches

- /matches
- /matches/[id]

Features:

- view matches
 - update score
-

Leaderboard

- /leaderboard/[id]
-

Profile

- /profile
-

UI COMPONENTS (REUSE THESE)

From your existing project:

- CourtCard → matches/tournaments
- GroupCard → clubs
- Navigation → layout

👉 Don't rewrite—reuse and adapt

API LAYER (IMPORTANT)

Create:

services/api.ts

Example:

```
import axios from "axios";
```

```
export const api = axios.create({
  baseURL: "http://localhost:8080",
});
```

Example API usage:

```
export const getTournaments = async () => {
  const res = await api.get("/tournaments");
  return res.data;
};
```

AUTH HANDLING

- Store JWT in:
 - localStorage (simple)
- Add interceptor:

```
api.interceptors.request.use((config) => {
  const token = localStorage.getItem("token");
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});
```

TEAM SPLIT (FRONTEND)

Person 1 — Auth + Layout

- login/signup pages
 - navbar
 - route protection
-

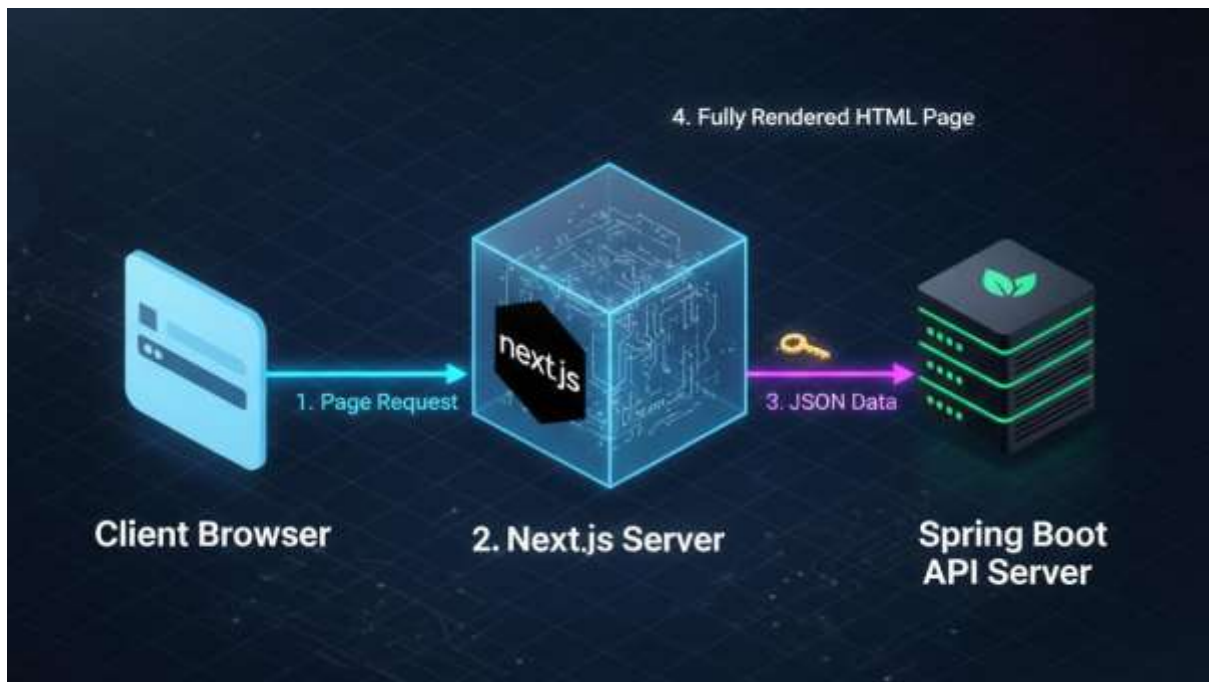
Person 2 — Clubs + Tournaments

- club pages
- tournament pages
- registration UI

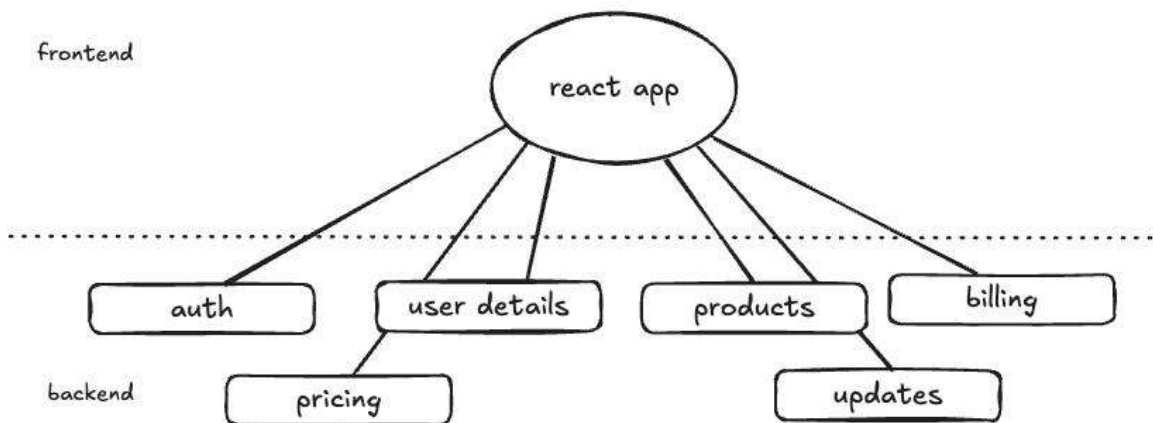
Person 3 — Matches + Leaderboard

- match pages
 - score UI
 - leaderboard UI
-

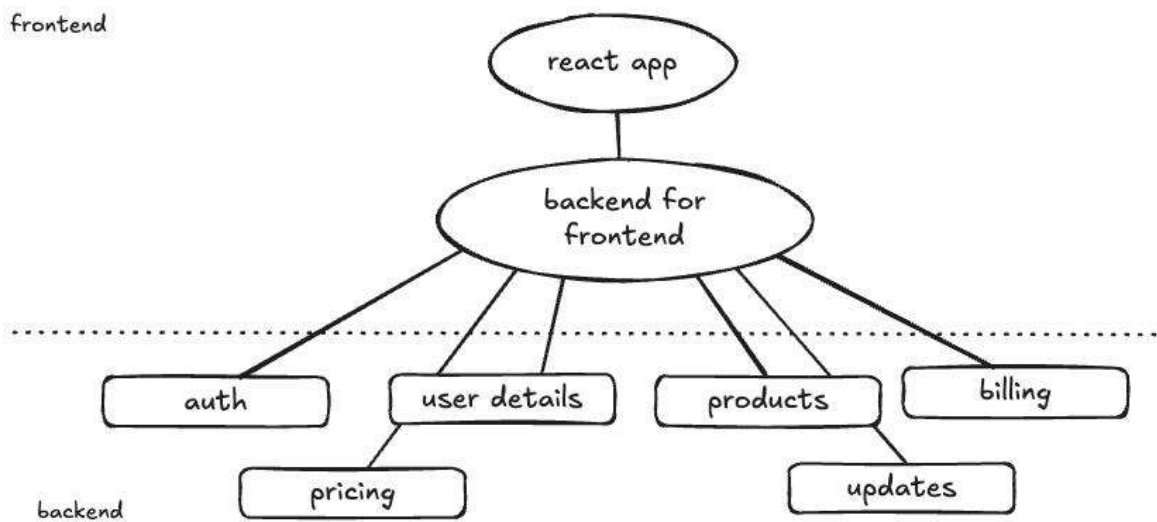
DATA FLOW



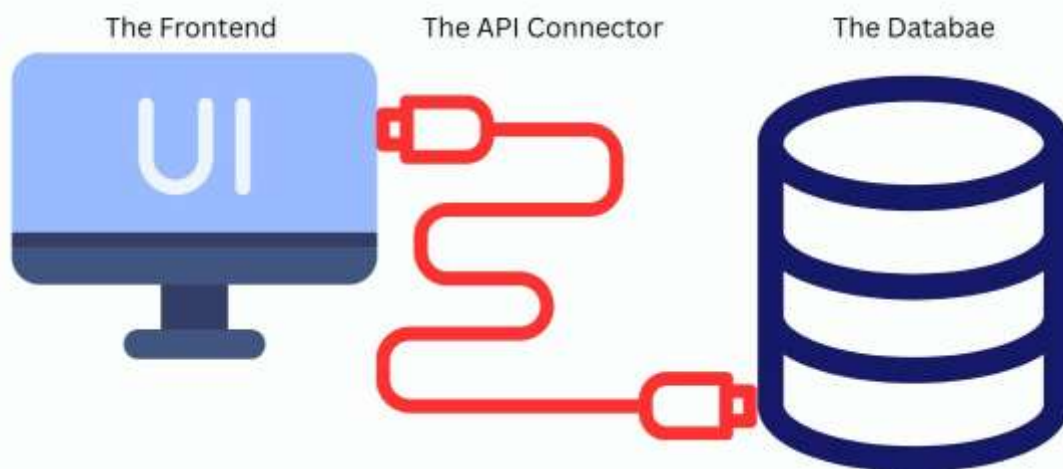
Standard - No BFF Used



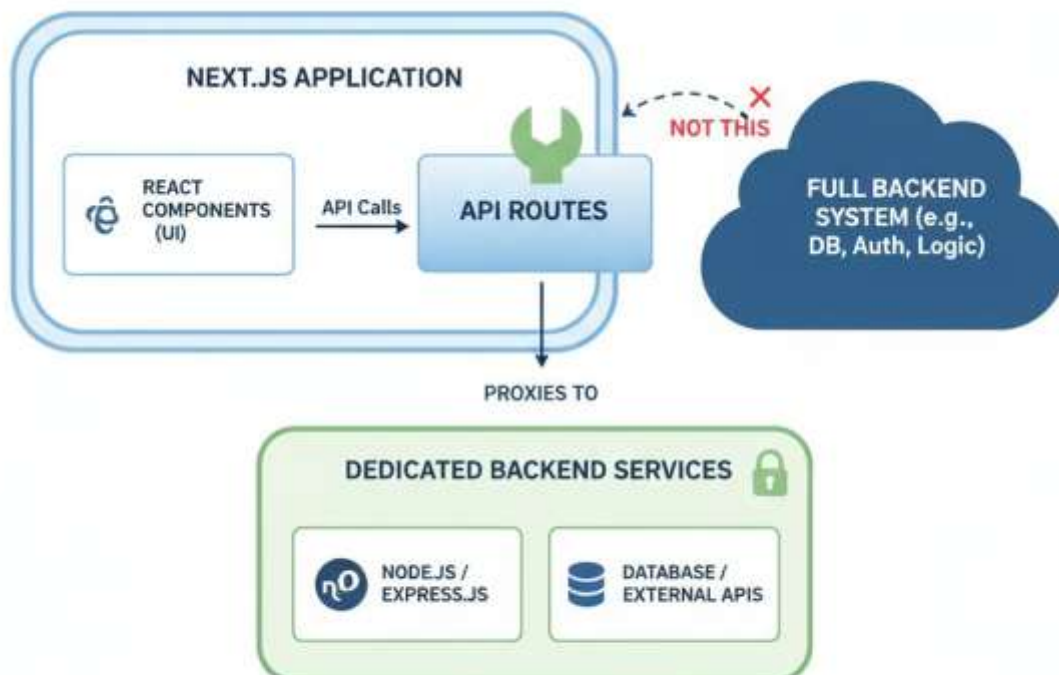
With a Backend For Frontend

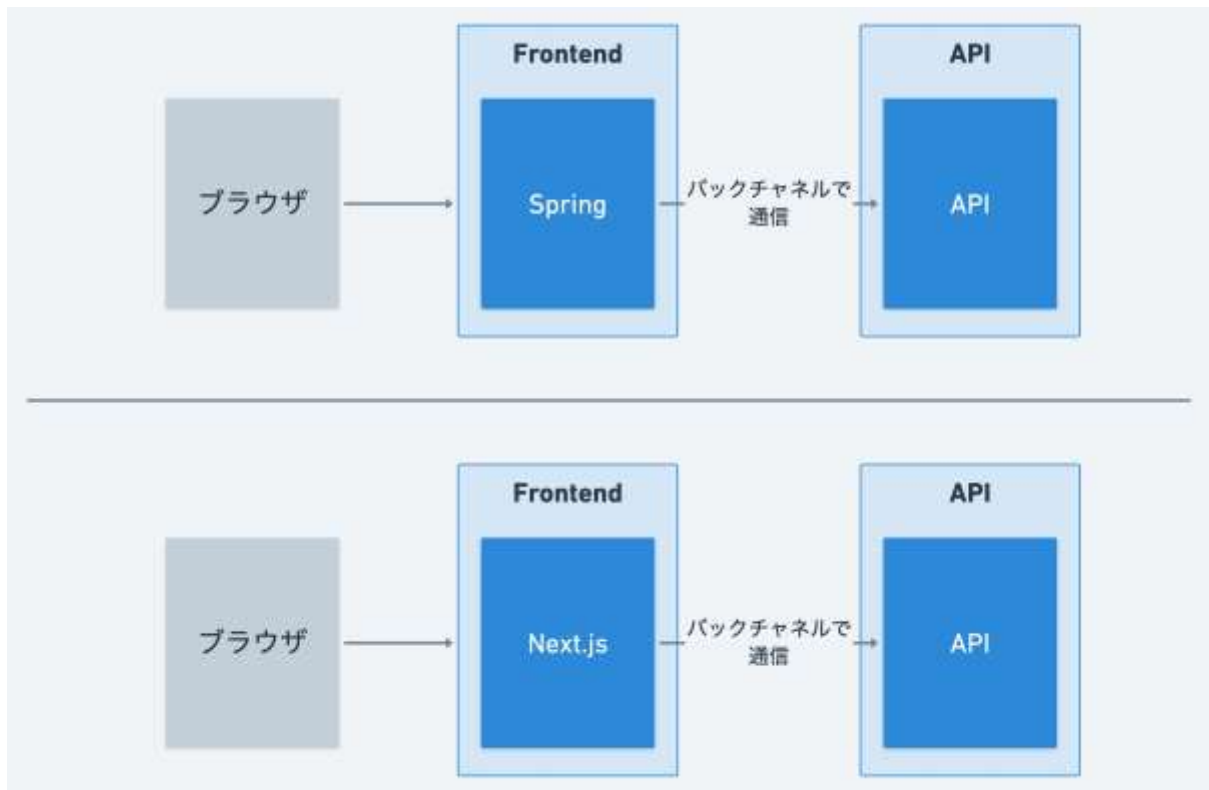


How to Integrate APIs into Frontend Applications: Writing a Reusable API Connector Like a Pro (Axios, Fetch & Next.js Server Actions)



NEXT.JS API ROUTES: Power & Perspective





User Action → Next.js UI → API call → Spring Boot → DB → Response → UI update

EXECUTION PLAN

WEEK 1

- Setup Next.js
- Create structure
- Auth pages
- Navigation

WEEK 2

- Clubs + tournaments UI
- API integration

WEEK 3

- Matches + leaderboard
- Full integration

🗪 WEEK 4

- UI polish
 - Bug fixing
 - Deployment
-

⚠️ RULES

- No hardcoded data
 - Use API layer only
 - Keep pages separate from components
 - Reusable components only
 - Keep UI simple first
-

🚫 COMMON MISTAKES

- ❌ Mixing logic in components
 - ❌ No folder structure
 - ❌ No API layer
 - ❌ Trying to perfect UI first
-

🧠 FINAL ARCHITECTURE

Next.js (Frontend)

↓

Spring Boot (Backend)

↓

PostgreSQL (Supabase)
